Transparent Multi-Hop Routing for Raft

Under Partial Network Partitions

Lorenzo Deflorian
Juozas Skarbalius

KTH · II2202

Partial partitions can interrupt Raft

Nodes may remain reachable through peers even when their direct links fail.

Consensus services need progress to remain available.

RQ1 Progress

When does forwarding restore stable commits?

RQ2 Latency

What delay does forwarding add to client-observed commits?

Goal: improve resilience while keeping the Raft protocol unchanged.

Raft and the routing gap

Raft commits after a majority stores an entry.
A five-node cluster needs three replicas.

Jensen et al. (2021)	Propose overlays for Raft
Nifty (2020)	Reroutes traffic in other systems
Omni-Paxos (2023)	Changes consensus for partial connectivity

Our gap: measure routing benefits and costs beneath unchanged Raft.

Controlled network experiments

900

validated trials

5 topologies
2 communication modes
3 link delays
30 repetitions

HashiCorp Raft in Mininet

Direct links vs. preconfigured forwarding

Link delay: 0.5, 5 or 20 ms
Heartbeat timeout: 500 ms
PreVote disabled

100 requests/s, 64-byte payloads

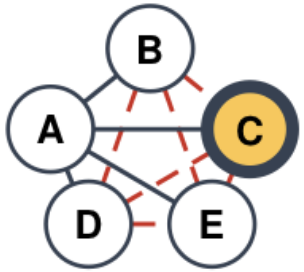
30 s warm-up, cut links, observe 30 s
T3: reseed and restart after injection

All 900 trials validated across the complete delay matrix.

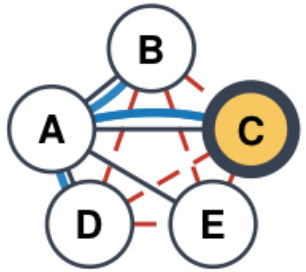
Partitions test quorum access and elections

T2 Quorum loss

Direct



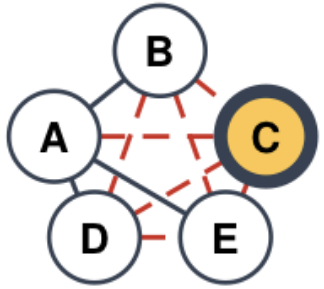
Forwarding



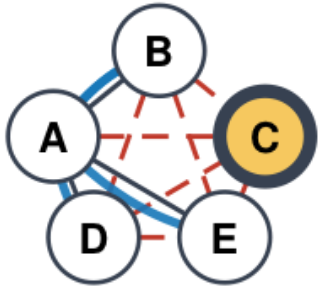
Leader C loses its direct majority.
A can relay traffic to other nodes.

T3 Constrained election

Direct



Forwarding



C is isolated. A has an older log.
Forwarding lets B reach a majority.

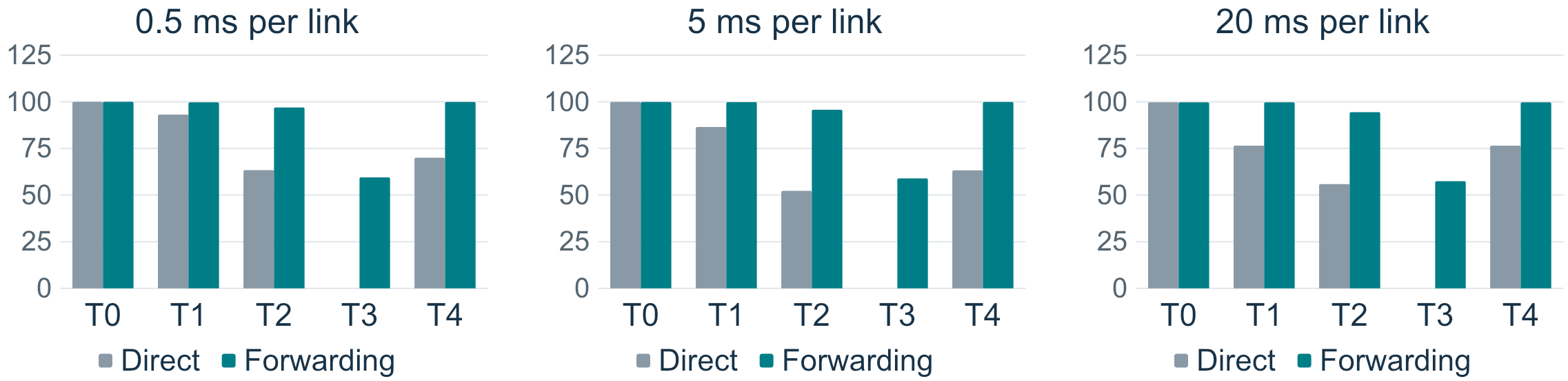
Solid links survive. Dashed links fail. Blue paths forward traffic.

T0: healthy control. T1 and T4: client reachability also matters.

Forwarding sustains client progress

Stable-progress heuristic: forwarding 450/450, direct 309/450

Mean successful client commits/s



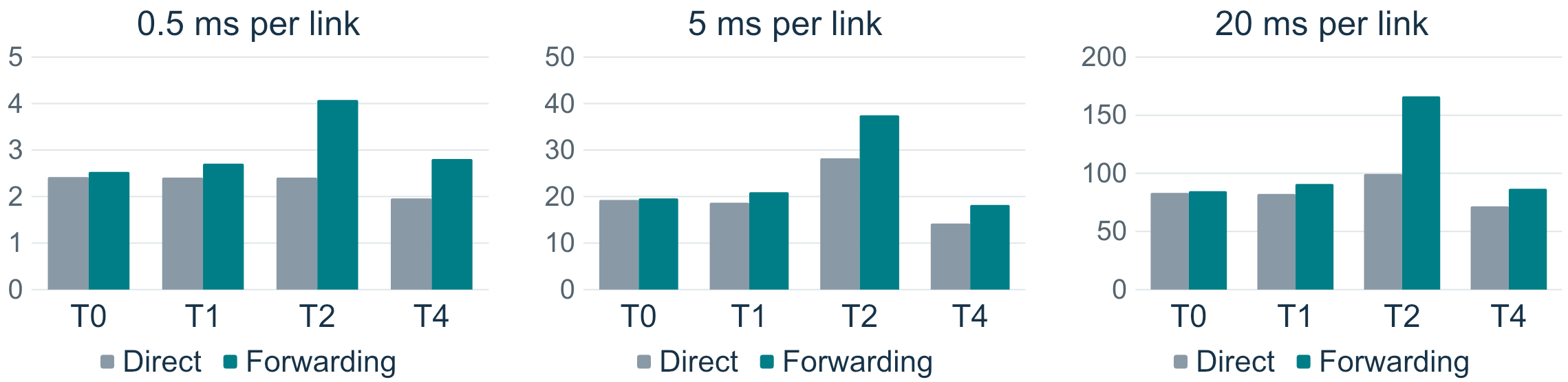
T2 throughput improves by 53%, 83% and 69% across the delays.

T3: all 90 forwarding trials commit. Direct mode commits in none.

n = 30 per condition. T3 throughput includes restart and setup time.

A forwarded quorum adds commit latency

Mean of per-run median successful-request latency (ms)



T2 direct / forwarding: 2.41 / 4.08, 28.23 / 37.49, 99.47 / 166.21 ms

T3 forwarding: 3.57, 35.81, 147.77 ms. No direct latency samples.

Axes use different scales. T1/T4 direct summaries exclude trials with no commits.

Routing can improve resilience at a cost

Progress

Forwarding restores T3 commits and improves T2 throughput.

Cost

Commits take longer when the majority needs extra hops.

T1/T4 gains also reflect client access to leaders that retain quorums.
T3 recovery includes setup. Results cover one workload and timeout.

Future work

01 Client reachability and recovery timing

Control client placement and separate setup from Raft recovery.

02 Heartbeat timeout and path delay

Vary the timeout to measure its interaction with network delay.

03 Deployment across machines and regions

Test variable delays in a more realistic network.

Questions

Transparent routing for Raft under partial partitions